DSA GRADED LAB

TASK 1: Marks 20

Scenario: Online Food Delivery System

You are developing a system for an online food delivery service. Orders are processed in the order they are received, but sometimes the system needs to **undo the last delivered order** due to cancellation or error.

Design and implement a program using **Stack and Queue** to manage food delivery orders.

👉 The system must follow these rules:

1. **Order Placement (Queue):**
   - New orders are added to a **Queue** (FIFO order).
   - Each order has an ID (e.g., O1, O2, O3).
2. **Order Delivery (Integration Logic):**
   - When an order is delivered:
     - Remove it from the **front of the queue**
     - Push it into a **Stack** (to track delivered orders)
3. **Undo Last Delivery (Stack):**
   - If a cancellation occurs:
     - Remove the last delivered order from the **Stack**
     - Add it back into the **Queue** (you must decide whether to insert at front or rear and justify your choice)

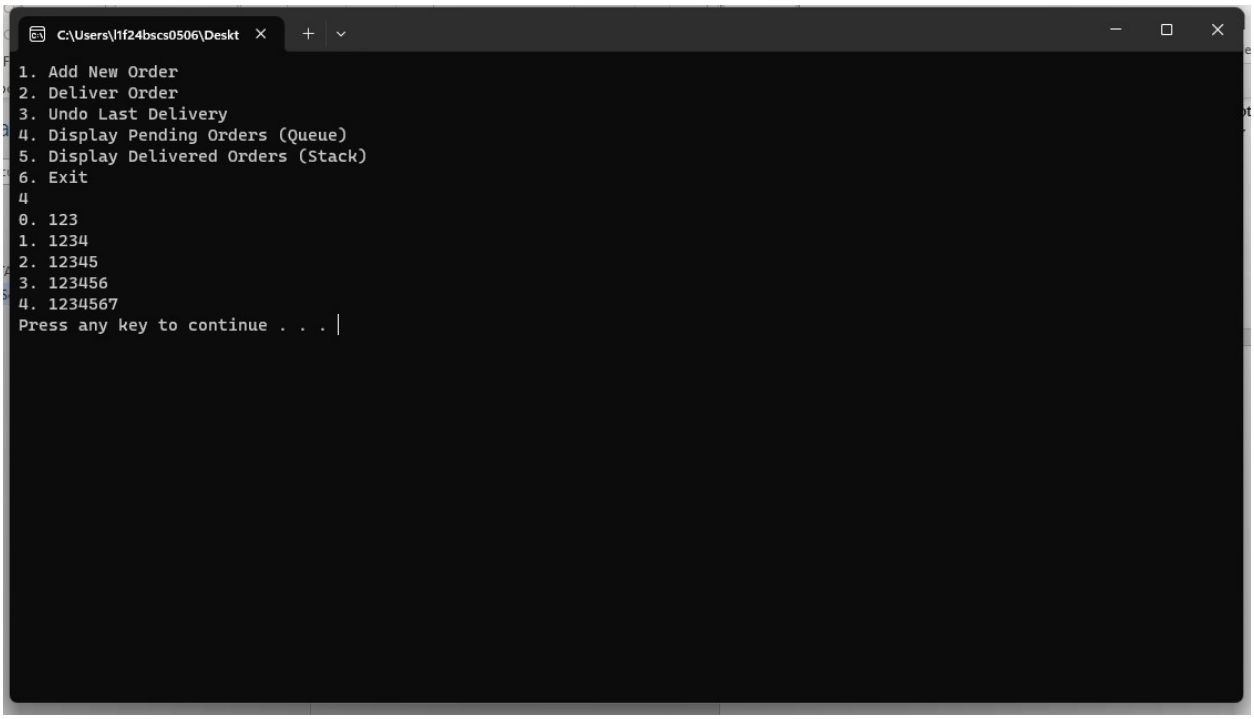
Create a **menu-driven program** with the following options:

1. Add New Order
2. Deliver Order
3. Undo Last Delivery
4. Display Pending Orders (Queue)
5. Display Delivered Orders (Stack)
6. Exit

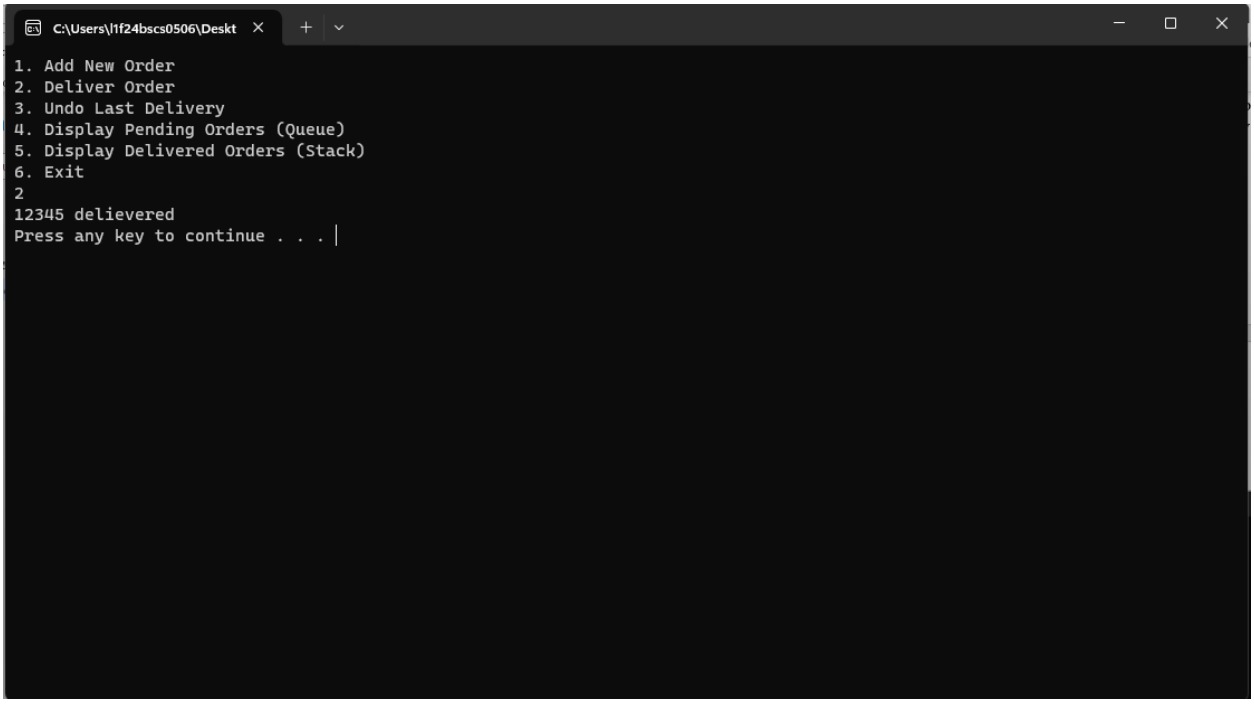
Limitations:

- Maximum queue size = 5
- Display `"Queue Full"` if no space is available
- Display `"No orders to deliver"` if queue is empty
- Display `"No delivery to undo"` if stack is empty

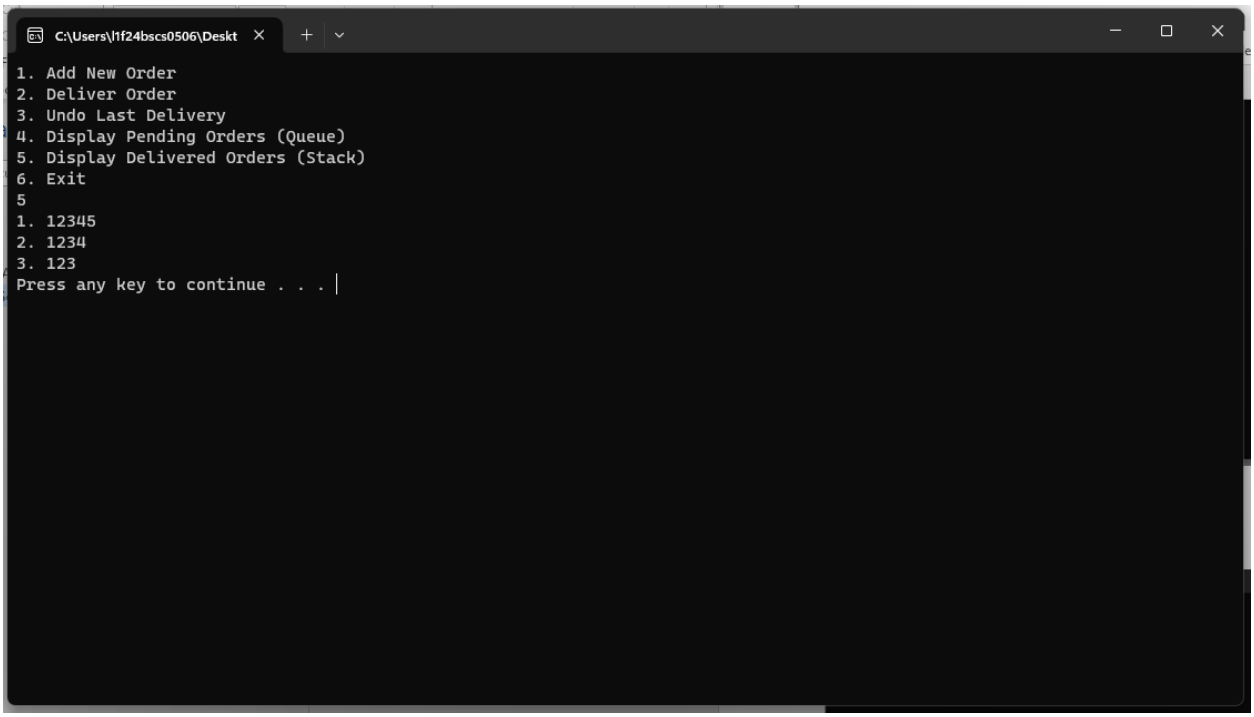
Orders in queue



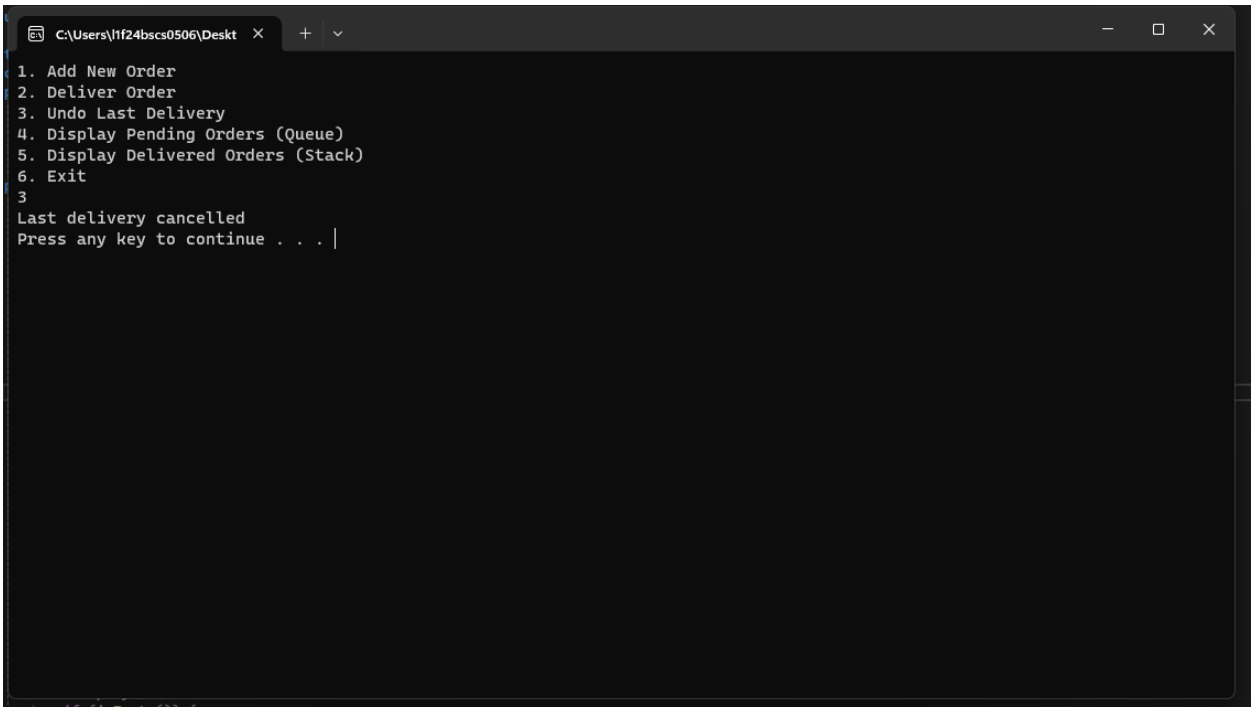
Stack Working



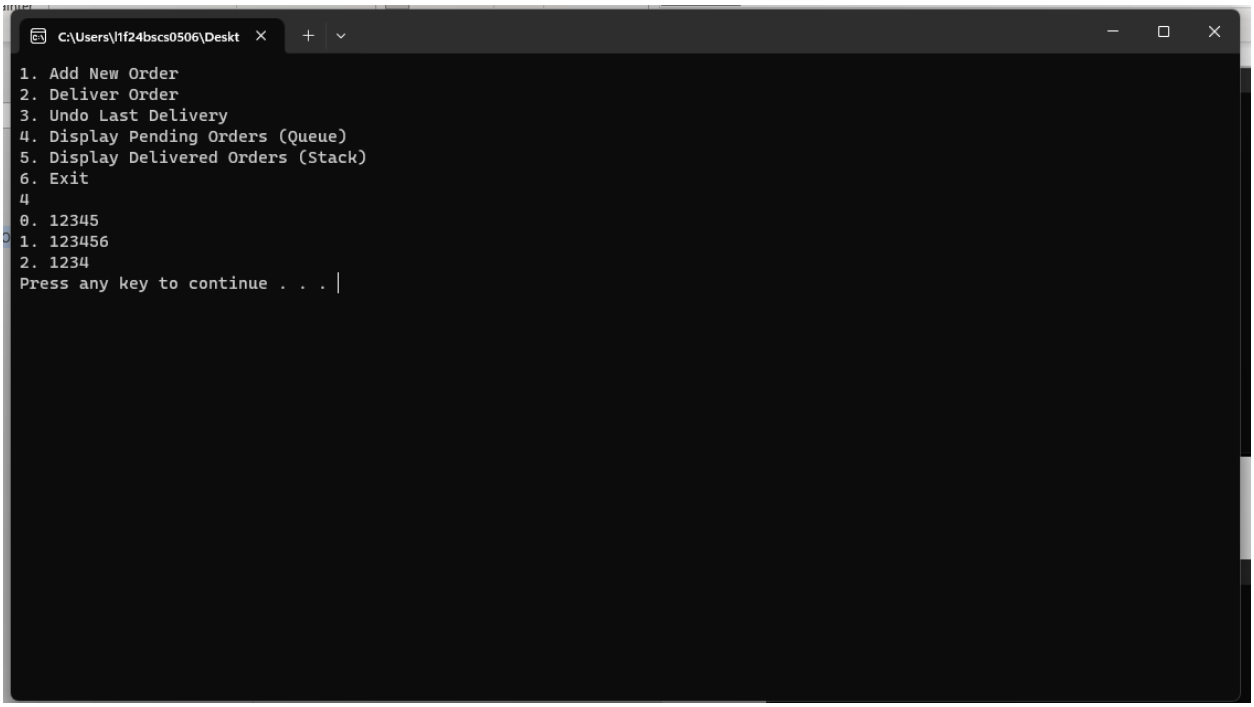
Delivered Orders



Cancelled Delivery



Added back to rear of queue



TASK 2:

Design a Smart Traffic Management System using Circular Queue. Implement all functions defined in the given header file. Emergency vehicles must be handled with priority insertion.

```
#include <iostream>
using namespace std;

#define SIZE 5 // Maximum vehicle

// Vehicle Structure (Scenario-based)
struct Vehicle {
int id;
string type; // "Normal" or "Emergency"
};

class TrafficQueue {
private:
Vehicle queue[SIZE];
int front;
int rear;

public:
// Constructor
TrafficQueue();

// Core Circular Queue Functions
void enqueueNormal(Vehicle v); // Add normal vehicle
void enqueueEmergency(Vehicle v); // Add emergency at priority position

void dequeue(); // Vehicle passes signal
void displayQueue(); // Show all vehicles

// Utility Functions
bool isFull();
bool isEmpty();

// Advanced Scenario Functions
void systemStatus(); // Show queue status report
int countEmergency(); // Count emergency vehicles
int countNormal(); // Count normal vehicles
};
```